Text-to-911 Middleware & Agent Console

Forward Deployed Engineer Candidate Exercise · RapidSOS

Context

Most 911 calls in North America are voice. But in growing numbers of situations (domestic violence, hearing-impaired callers, satellite-only coverage in remote areas), voice isn't an option. RapidSOS works with carriers and emergency call centers to enable text-to-911 in places it wasn't previously available.

At a high level, the pattern looks like this: a carrier's SMS infrastructure receives a text from a user to 911 (or a designated short code). Instead of dropping it, the carrier forwards it to a middleware service. That service maintains the session, surfaces the conversation to an agent in real time, lets the agent reply, and handles the unglamorous lifecycle problems like session timeouts, expired sessions, retries, message ordering.

You're going to build a simplified version of that middleware, plus a basic agent console to interact with it.

The Scenario

A fictional carrier (call them Northstar Telecom) is launching a text-to-911 capability for their subscribers, with us as the platform partner. Their network team has shared two API contracts:

- **Inbound webhook (Northstar → us):** when a user texts 911, Northstar POSTs the message to our service.
- **Outbound API (us → Northstar):** when an agent sends a reply, we POST to Northstar, who delivers the SMS to the user's handset.

Sessions are stateful. The first inbound message from a phone number opens a session; subsequent messages from that number within the session window belong to that session. After 5 minutes of inactivity, the session expires and any reply from the agent should be rejected. The user texting in again after expiry should start a new session.

What You'll Build

1. A middleware service

A backend (any language, but Python or Node strongly preferred — those are what most of our codebase uses) that exposes:

- **`POST /inbound`** — receives messages from "Northstar". Body: `{ "from": "+15551234567", "text": "...", "timestamp": "..." }`.
- **`GET /sessions`** — returns the list of currently-active sessions (for the agent UI to poll, or use SSE/WebSocket if you prefer).
- **`GET /sessions/{id}`** — returns the full message history for one session.
- **`POST /sessions/{id}/reply`** — agent sends a reply. Your service should call the (mock) Northstar outbound API.

Your service is responsible for:

- Session lifecycle: open on first message, refresh on each message, expire after 5 minutes of inactivity
- Rejecting agent replies to expired sessions with a clear error
- Idempotent inbound handling — if Northstar retries the same message, you don't duplicate it in the session
- Reasonable handling of the outbound API failing (what does the agent see? what gets retried?)

2. A mock "Northstar" outbound endpoint

You don't have a real carrier. Stand up a trivial stub endpoint that pretends to be Northstar's outbound delivery API — accepts your POSTs, logs them, returns 200 (or, occasionally, a 5xx, so we can see how your middleware handles delivery failure). Should be ~30 lines.

3. An agent console

A web UI that an agent uses during a session. It should:

- Show all active sessions in a list, with the most recent message preview and timestamp
- Let the agent click into a session and see the full conversation
- Provide a text input to reply, with feedback when the reply succeeds or fails
- Update in near-real-time as new inbound messages arrive (polling is fine; SSE/WebSocket is better)
- Make session expiry visible — the agent should know when a session has expired and replies will be rejected

No design system required. Clarity beats polish. The agent is doing this under pressure; the UI should be legible at a glance.

4. A test harness

A small script that simulates a carrier sending a sequence of messages — at least:

- A short conversation that resolves normally
- A conversation that expires (no activity for >5 min) and the agent tries to reply
- A duplicate inbound delivery (same message twice) to verify idempotency
- Two parallel sessions from different phone numbers

We'll run this during your presentation, so make it easy to invoke.

What We're Not Asking For

To keep this scoped to ~4–6 hours, please skip:

- Authentication / authorization (just trust requests)
- Persistent storage (in-memory is fine — call this out in the README)
- Production deployment (running locally is fine)
- Visual design beyond legibility
- Phone number validation or carrier-correctness
- Anything to do with real PSAPs, real geographic routing, or real 911 protocols

If you find yourself reaching for any of these, note it in the README as something you'd build next and move on.

On Ambiguity

This brief deliberately leaves some things underspecified. That's on purpose — FDE work always does. A few examples you'll probably hit:

- What happens if a user texts during the session expiry grace period — does it extend the session or start a new one? You decide; defend the choice.
- Should the agent see expired sessions in the list? For how long?
- What's the right retry policy when the outbound API returns 5xx?

We're more interested in how you reason about these than in the specific answer. Document your decisions in the README.

On AI Tools

Use them. Claude, Cursor, Copilot, ChatGPT, Replit, Lovable — there is no penalty and no expectation that you write every line by hand. At RapidSOS, our FDEs use AI tooling daily and we want to see how you use it.

What we do care about:

- You can defend every architectural decision, including ones the AI suggested
- You can walk us through what each non-trivial piece of code is doing and why
- You can explain what would change if this went to production

In the presentation we'll pick a couple of files and ask: "walk me through this. Why this approach?" If the answer is "the AI wrote it that way," that's a signal we want to dig into.

How We'll Evaluate

Dimension	What we're looking for
System design	Session state, idempotency, and failure handling are thought through. Reasoning shows up clearly in the code or README.
Code quality	Readable, sensibly structured, easy to extend. Tests or at least a runnable harness exist.
Edge case instincts	You found and addressed the gotchas — expired sessions, duplicate deliveries, outbound failures — without being asked.
Product judgment	The agent UI prioritizes what matters under pressure. You can articulate why.
Communication	You can explain your design to a non-engineer (the "carrier PM" persona) and an engineer (the "carrier network team" persona) in the same conversation.

AI fluency	You used AI tools effectively but understand what you shipped.

Logistics

- **Time budget:** plan for 4–6 hours total. If you find yourself going longer, ship what you have and use the README to describe what you'd cut/add.
- **Deliverable format:** a GitHub repo (public or shared private) with code, README, and a Makefile or one-line command to run everything locally.
- **Submission deadline:** [to be confirmed at scheduling]
- **Presentation:** 30-minute presentation + live walkthrough (with us running your test harness against your service) + 15 minutes of Q&A.
- **Questions during the exercise:** email is fine. We'll respond same-day. There are no trick questions; if something is ambiguous, ask — or make a reasonable assumption and document it.

One Last Thing

This brief is intentionally a little open. Real FDE work always is — a customer hands you a messy problem with half-defined APIs, and you decide what to build. We care more about how you scope and reason than whether you tick every box on this page. If you build something we didn't ask for that demonstrates judgment, that's a win.

Good luck — we're looking forward to seeing what you build.